Topic: Terraform Advanced Commands — Taint, Import, Workspace, Refresh, Logging & User Data

(Your notes, cleaned up from dictation + screenshots — Lecture 3)

Quick recall

With the basics covered (workflow, state, references), this session moved on to the more advanced CLI topics from the lecture's list:

Command / feature	One-liner
`terraform graph`	Visual representation of resources and how they depend on each other *(covered in Chapter 02)*
`terraform taint`	Manually mark a resource as **damaged / needing replacement** on the next `apply`
`terraform import`	Bring **existing infrastructure** (not created by Terraform) under Terraform's management
`terraform workspace`	Manage **multiple deployments** (dev / UAT / prod) of the **same configuration**
`terraform refresh`	Read current settings from real infra and **update the state file** to match
Terraform logging	Record the **internal steps** Terraform takes (`TF_LOG`)
Provisioners	Run scripts/commands on a local or remote machine during provisioning *(not covered yet)*
Plugin cache	Local storage for provider plugins, so `init` is faster *(not covered yet)*
User data	Instructions you give a new server to **run automatically at startup**

---

1. `terraform taint` — force a resource to be replaced

What "tainted" means

If we mark a resource as **tainted**, we are telling Terraform: "*this particular resource is damaged, replace it on the next* `terraform apply` ." On that next apply Terraform **destroys** the resource and **creates** a fresh one.

Hands-on

We had two resources in `project1`:

```
terraform state list
# aws_instance.web
# aws_s3_bucket.my_s3_bucket
```

Taint the EC2 instance:

```
terraform taint aws_instance.web
# Resource instance aws_instance.web has been marked as tainted.
```

The state file changes **immediately**. Open `terraform.tfstate` and the instance now has:

```
"instances": [
  {
    "status": "tainted",
    "schema_version": 2,
    "attributes": { "ami": "ami-003c4e453d6057f7d", ... }
```

Changed your mind (the team says "no, we still need this machine")? Undo it:

```
terraform untaint aws_instance.web
# Resource instance aws_instance.web has been successfully untainted.
```

Taint it again and apply:

```
terraform taint aws_instance.web
terraform apply --auto-approve
# ...
# Plan: 1 to add, 0 to change, 1 to destroy.
```

`1 to add, 1 to destroy` = a **replacement**. After the apply, the new instance's entry in the state file no longer has `"status": "tainted"`.

Default order is destroy first, then create. You can flip it to *create the new one first, then destroy the old one* with the `lifecycle` block:

```
resource "aws_instance" "web" {
  # ...
  lifecycle {
    create_before_destroy = true
  }
}
```

Why taint is risky — the "ticking time bomb"

`taint` writes to the **shared** state file right now, but nothing actually happens until *someone* runs the next `apply`. In between, another team might be doing important work on that EC2 instance, with no idea it's already scheduled to be destroyed. Whoever runs the next `apply` (maybe for a completely unrelated change) wipes it out. That's why this approach **should not be used in production**.

The modern way: `terraform apply -replace`

```
terraform apply -replace="aws_instance.web"
```

Capability	Legacy (`terraform taint`)	Modern (recommended)
Command syntax	`terraform taint <RESOURCE_ADDRESS>`	`terraform apply -replace="<RESOURCE_ADDRESS>"`
State file impact	Modifies the state file **immediately**	Only modifies the state **during execution**
Execution risk	**High** — creates a hidden "ticking time bomb" in shared state	**Low** — safe for simultaneous, collaborative environments
Status	**Deprecated** since v0.15.2	Fully supported in stable versions

> **Note:** `-replace` does the replacement *in that same run*, so nothing is left hanging in state for someone else. It also shows up in the plan (`-/+ must be replaced`), so you can review it first. `terraform plan -replace="aws_instance.web"` previews it without touching anything.

---

2. `terraform import` — bring existing infra under Terraform

Why we need it

Say a company has been running on AWS for years and built a lot of infrastructure **manually through the console**. Now management decides: from here on, everything goes through an IaC tool, Terraform. New resources will be created with Terraform. But what about the existing ones?

If we leave them as they are and keep updating them through the console, while new stuff comes from Terraform, that's **chaos**: two sources of truth. We need to pull all the existing infrastructure **under the roof of Terraform**. That's where `terraform import` comes in.

> It maps real-world infrastructure resources into your Terraform **state file** so Terraform can manage them going forward.
>
> ⚠️ **Important:** it does **not** automatically generate the corresponding `.tf` configuration. You must write that yourself (or use a tool to generate it).

Syntax:

```
terraform import [options] ADDRESS ID
# ADDRESS — Terraform resource address, e.g. aws_instance.example
# ID      — the resource's ID in the provider's platform, e.g. the EC2 instance ID
```

Hands-on: import a console-created EC2, then change its instance type via Terraform

Task: an EC2 instance `Terraform-import-ec2` (`i-0c27f664619a950cc` , `t2.nano` , ap-south-1b) was created **manually through the console**. We received a task to change its instance type, but **not manually**. It has to be done through Terraform.

Step 1 — write an empty resource block as the landing spot for the import:

```
resource "aws_instance" "terraform-import" {
}
```

> **Gotcha:** in one screenshot the file was saved as `ec2-tf` (hyphen), not `ec2.tf` . Terraform only reads files ending in `.tf` , so `import` can't find the resource address in a wrongly named file. It's `ec2.tf` by the time the import succeeds.

Step 2 — import it:

```
terraform import aws_instance.terraform-import i-0c27f664619a950cc
# aws_instance.terraform-import: Import prepared!
#   Prepared aws_instance for import
# Import successful!
# The resources that were imported are shown above. These resources are now in
# your Terraform state and will henceforth be managed by Terraform.
```

Step 3 — verify it's in state:

```
ls -l                # ec2.tf (48 bytes) + terraform.tfstate (5477 bytes) — state now popul
terraform state list  # aws_instance.terraform-import
terraform show        # every attribute imported: ami, arn, availability_zone = "ap-south-1b
```

That's **50% of the task done**. The resource is in the **state**, but `ec2.tf` is still an empty block. If we `apply` now:

```
Error: "launch_template": one of `ami,instance_type,launch_template` must be specified
Error: "ami": one of `ami,launch_template` must be specified
```

The config and reality don't match. That's **drift**, and there are two ways to fix it:

1. **Manual:** write the `.tf` config yourself (what we did below).
2. `import` **block:** the declarative way (see the note further down).

Step 4 — reconcile the drift. Add the mandatory fields (values taken from `terraform show`), then `terraform fmt ec2.tf` to tidy the alignment:

```
resource "aws_instance" "terraform-import" {

  ami           = "ami-066c4849e6b3a1e3d"
  instance_type = "t2.nano"
  key_name      = "Terraform-import"

}
```

`terraform plan` → **no error, but also not "No changes"**:

```
~ resource "aws_instance" "terraform-import" {
    ~ tags     = { - "Name" = "Terraform-import-ec2" -> null }
    ~ tags_all = { - "Name" = "Terraform-import-ec2" -> null }
    + user_data_replace_on_change = false
  0 to add, 1 to change, 0 to destroy.
```

Terraform wanted to **remove the Name tag**, because our config didn't mention it. We'd missed the `tags` attribute. Add it:

```
  tags = {
    Name = "Terraform-import-ec2"
  }
```

`terraform plan` again:

```
No changes.
Terraform has compared your real infrastructure against your configuration and
found no differences, so no changes are needed.
```

That's the goal state. Keep filling in properties from reality until `plan` says *No changes*. Only then is the import truly complete.

> **Lesson:** "plan ran without error" ≠ "import is done". Read the plan: any `~` or `-` means your config still disagrees with reality, and the next apply would change the real resource.

Step 5 — now make the actual change (`t2.nano` → `t2.micro`):

```
  instance_type = "t2.micro"
```

```
terraform apply -auto-approve
# ~ resource "aws_instance" "terraform-import" {
#     id            = "i-0c27f664619a950cc"
#   ~ instance_type = "t2.nano" -> "t2.micro"
#   ~ public_dns    = "ec2-13-203-214-50.ap-south-1.compute.amazonaws.com" -> (known after a
#   ~ public_ip     = "13.203.214.50" -> (known after apply)
# 0 to add, 1 to change, 0 to destroy.
# Apply complete! Resources: 0 added, 1 changed, 0 destroyed.
```

The console confirms: **Instance type** `t2.micro`, same instance ID.

> **Worth noticing:** it's an *update in-place* (same instance ID, not a replacement), but the **public IP changed** from `13.203.214.50` to `13.235.75.134`. AWS has to **stop and start** an instance to change its type, and a non-Elastic public IP is released on stop. The plan warned us with `public_ip -> (known after apply)`. In-place ≠ no downtime.

The `import` block (Terraform 1.5+)

Instead of the CLI command, you can declare the import in code:

```
import {
  to = aws_instance.web_server
  id = "i-0123456789abcdef0"
}
```

> **Note:** this is actually the **modern, recommended** way, not just for testing. It goes through the normal `plan` → `apply` flow (so it's reviewable). And it can **write the config for you**:
>
> ```
> terraform plan -generate-config-out=generated.tf
> ```

> That fills in the attributes Step 4 had us type by hand. Review and clean the generated file, then `apply`. Once the import has happened, the `import` block can be removed.

---

3. `terraform workspace` — one config, multiple isolated deployments

> **Terraform workspaces** allow you to manage **separate, isolated state files** for different environments (like `dev`, `staging`, or `prod`) using the **exact same configuration files**.

The idea

We keep our configuration file in GitHub. Different teams (Dev, UAT, Production) all use that same configuration, each for their own needs:

- **Dev:** it's just dev, so a big instance isn't necessary: `t2.micro` / `t2.nano`.
- **UAT:** some load testing happens, so a different instance size.
- **Prod:** actual production-grade instances.

We don't rewrite the configuration again and again for each team. **Same code, different workspace, separate state.**

Commands

```
terraform workspace            # help: new, list, show, select, delete
terraform workspace new Dev-team
# Created and switched to workspace "Dev-team"!
# You're now on a new, empty workspace. Workspaces isolate their state,
# so if you run "terraform plan" Terraform will not see any existing state
# for this configuration.
terraform workspace new UAT-team
terraform workspace new Prod-team

terraform workspace list       # * marks the active one
#   default
#   Dev-team
# * Prod-team
#   UAT-team

terraform workspace show       # Prod-team  (just the current one)
terraform workspace select Dev-team
# Switched to workspace "Dev-team".
terraform workspace delete <name>
```

Where the state lives: `terraform.tfstate.d/`

Creating a workspace automatically creates a directory `terraform.tfstate.d/`. Terraform keeps **each workspace's state file** in its own subfolder. When I work in Dev, only Dev's state changes; in UAT, only UAT's; in Prod, only Prod's. That's the isolation.

```
terraform.tfstate.d/
├── dev-team
│   └── terraform.tfstate
├── Dev-team
│   └── terraform.tfstate
├── Prod-team
└── UAT-team
```

Observations from this tree:

- `Prod-team` and `UAT-team` are **empty**. A workspace's state file only appears after its first `apply`.
- `dev-team` and `Dev-team` are **two different workspaces**. Workspace names are **case-sensitive**, so one typo in `select` / `new` gives you a whole separate environment.
- The `default` workspace doesn't live here. Its state stays at `./terraform.tfstate` in the project root.

`${terraform.workspace}` — make resources workspace-aware

`terraform.workspace` is a built-in value that returns the **current workspace name**. Append it to names so you can tell which team/environment created what:

```
resource "aws_s3_bucket" "example" {
  bucket = "my-tf-test-bucket-0788-${terraform.workspace}"

  tags = {
    Name = "My bucket-${terraform.workspace}"
  }
}
```

In the `Dev-team` workspace, `terraform apply` → `1 added` → bucket `my-tf-test-bucket-0788-Dev-team`, and its state is written under `terraform.tfstate.d/Dev-team/`.

> **Note:** this is also how the "Dev gets `t2.micro`, Prod gets something bigger" idea is done without editing the file per team. Key a map off the workspace:
>
> ```
> instance_type = lookup({ "Dev-team" = "t2.micro", "UAT-team" = "t2.small", "Prod-team"
>                          terraform.workspace, "t2.micro")
> ```

> (S3 bucket names must be lowercase, so a mixed-case workspace like `Dev-team` inside a bucket name is risky. `lower(terraform.workspace)` avoids that.)

---

4. `terraform refresh` — sync state with reality

> `terraform refresh` **reads the current settings** from all managed remote objects and **updates the Terraform state** to match.

If someone changed something **manually** (drift), running refresh pulls the current real values into the state file:

```
terraform refresh
# aws_instance.linux: Refreshing state... [id=i-048ef6b5b8412181a]
```

But it's been deprecated by HashiCorp, because it **modifies the state instantly and silently**, without letting you preview or approve the changes. There's no chance to say yes or no, which is dangerous.

The replacement is the `-refresh-only` mode:

```
terraform plan  -refresh-only    # PREVIEW: what would change in state, touches nothing
terraform apply -refresh-only    # show the same diff, ask for approval, then update state
```

> **Remember:** refresh (either form) only updates the **state file**. It never changes real infrastructure and never edits your `.tf` files. And a normal `plan` / `apply` already refreshes implicitly first.

---

5. Terraform logging — `TF_LOG`

Terraform logging **records the internal steps** Terraform takes when managing your infrastructure. By default it's **off**. Turn it on with the `TF_LOG` environment variable.

Levels, least → most verbose:

Level	What you get
`OFF`	Logging off (default)
`ERROR`	Only the least verbose output, typically user-facing errors
`WARN`	Also warnings about deprecations or potential issues
`INFO`	High-level messages about the execution process
`DEBUG`	More detailed information, useful for general debugging
`TRACE`	Most verbose: low-level execution steps and internal logs (recommended when reporting bugs)

Send it to a file with `TF_LOG_PATH`. We added both to `~/.bashrc`:

```
export TF_LOG="TRACE"
export TF_LOG_PATH="./terraform.log"
```

```
vim /root/.bashrc
source /root/.bashrc
terraform plan
ls -ltrha           # terraform.log (2.9K) appears
less terraform.log
# 2026-09-24T13:49:54.444Z [INFO]  Terraform version: 1.16.3
# 2026-09-24T13:49:54.444Z [DEBUG] using github.com/hashicorp/hcl/v2 v2.24.0
# 2026-09-24T13:49:54.444Z [INFO]  Go runtime version: go1.26.8
# 2026-09-24T13:49:54.444Z [INFO]  CLI args: []string{"terraform", "plan"}
# 2026-09-24T13:49:54.444Z [TRACE] Stdout is a terminal of width 79
```

> **Heads-up:** putting it in `.bashrc` makes **every** Terraform command log at TRACE from now on. That's noisy, the file keeps growing, and TRACE logs can contain sensitive values. For day-to-day work, set it just for one run (`TF_LOG=DEBUG terraform plan`) or `unset TF_LOG TF_LOG_PATH` when you're done. Also, `./terraform.log` is relative, so it lands in whichever directory you run Terraform from. `TF_LOG_CORE` / `TF_LOG_PROVIDER` split Terraform-core vs provider-plugin logs.

---

6. User data — run a script when the server boots

> Terraform user data is a kind of instructions you give a new server to **run automatically at startup**.

Same idea as **User data** in the AWS EC2 console: we pass commands or a script, and when the instance launches, those commands get executed. In Terraform we pass it through the resource's `user_data` argument.

The script (`ram.sh` , sitting next to `main.tf`):

```
#!/bin/bash
sudo yum install httpd -y
sudo systemctl start httpd
sudo systemctl enable httpd
echo "sample webserver using terraform" > /var/www/html/index.html

sudo hostnamectl set-hostname web_server
```

The resource, reading the script in with the `file()` function:

```
resource "aws_instance" "ourfirst" {
  ami               = "ami-0b41f7055516b991a"
  availability_zone = "ap-south-1a"
  instance_type     = "t2.micro"
  key_name          = "ram_aws"
  user_data         = "${file("ram.sh")}"
  tags = {
    Name = "test_mahcine"
  }
}
```

On `apply` , the instance boots, runs `ram.sh` , and comes up already serving the sample web page.

> **Notes:**
>
> - `"${file("ram.sh")}"` is the old 0.11-style wrapper. Today just write `user_data = file("ram.sh")` (or `file("${path.module}/ram.sh")` so the path doesn't depend on where you run Terraform).
> - User data runs **once, on first boot** (via cloud-init). Editing the script later and applying won't re-run it on the existing instance, unless you set `user_data_replace_on_change = true` , which **replaces** the instance. (That's the `user_data_replace_on_change = false` line in the import plan above.)

Command summary (this chapter)

```
# Taint / replace
terraform taint <addr>                        # legacy — mark for replacement on next apply (
terraform untaint <addr>                      # undo a taint
terraform apply -replace="<addr>"             # modern — replace in this run
terraform plan  -replace="<addr>"             # preview the replacement

# Import
terraform import <addr> <provider-id>         # e.g. aws_instance.terraform-import i-0c27f664
terraform plan -generate-config-out=gen.tf    # with an import {} block: generate the config
terraform show                                # read imported attributes to copy into .tf
terraform fmt ec2.tf                          # tidy the hand-written config

# Workspaces
terraform workspace new|list|show|select|delete <name>
# ${terraform.workspace} in HCL -> current workspace name

# Refresh
terraform refresh                             # deprecated — silently rewrites state
terraform plan  -refresh-only                 # preview drift
terraform apply -refresh-only                 # accept drift into state, with approval

# Logging
export TF_LOG=TRACE                           # OFF|ERROR|WARN|INFO|DEBUG|TRACE
export TF_LOG_PATH=./terraform.log
unset TF_LOG TF_LOG_PATH
```

Still to come (from the lecture's list)

- **Provisioners** — running scripts on local/remote machines during provisioning (`local-exec`, `remote-exec`, `file`).
- **Plugin cache** — sharing downloaded providers across projects to speed up `init`.